# 3D Dungeon Escape
# Maze Adventure

Engine: PyOpenGL (OpenGL + GLUT + GLU)
Language: Python

# Project Overview

3D Dungeon Escape is a  dungeon crawler developed using Python and the PyOpenGL library. The game places the player inside a procedurally generated 3D maze where they must navigate through corridors, combat enemies, interact with environmental objects such as doors and levers, collect useful items, and ultimately reach the exit point to complete the level.

The project demonstrates several fundamental and advanced computer graphics concepts, including real-time rendering, geometric transformations, collision detection, and the integration of 2D UI overlays within a 3D environment.

# Technology Stack

| Component | Details |
|---|---|
| Programming Language | Python 3 |
| Graphics API | OpenGL (PyOpenGL) |
| Windowing | GLUT |
| Utilities | GLU |
| Libraries | PyOpenGL |
| Data Handling | JSON (Save/Load System) |

# Game Design

## Concept

The player begins at a starting position within a dungeon and must reach an exit placed at the farthest possible location. The environment is interactive and dynamic, featuring locked and unlocked doors, levers that control pathways, hostile enemies, and collectible items such as keys and health packs.

# Features:

| Serial | Feature |
|---|---|
| 1 | WASD movement with arrow keys look around |
| 2 | Health, stamina, and inventory system |
| 3 | Door and lever interaction |
| 4 | Sword and crossbow combat |
| 5 | Procedural dungeon generation |
| 6 | Head-bob camera effect |
| 7 | Multiple enemy types |
| 8 | Health bars and damage flash |
| 9 | Hit detection with cooldown |
| 10 | Boss phase changes |
| 11 | Enemy collision |
| 12 | Minimap with exploration |
| 13 | Level progression and exit |
| 14 | Save and load game state |
| 15 | 3rd Person view of over-all game |

# Features by Member

## Member 1: Minhaz (Core Mechanics & Player)

I was responsible for implementing the responsive WASD movement with smooth collision detection to prevent passing through walls. I also developed the health, stamina, and inventory system, allowing players to collect keys and health packs. The dual-weapon system (sword and crossbow) was implemented with different damage values and attack speeds. Additionally, I created the object interaction system, enabling players to open doors, pick up items, and use levers. Finally, I developed the procedural dungeon generation algorithm that creates unique rooms and corridors using random seeds for each level.

## Member 2: Farhan (Enemies & Combat)

I focused on designing the head-bob camera as a first person view mode. I implemented enemy AI with pathfinding, allowing enemies to move toward the player using simple steering behavior. I introduced multiple enemy types (melee, ranged, and fast), each with unique movement speeds and attack patterns. Visual combat feedback was added through health bars above enemies and a red flash effect when enemies are damaged. I also developed the hit detection and cooldown system for both ranged and melee attacks. A boss enemy was implemented, which appears every 3 levels and features special attack patterns and a second phase when its health drops below 50%.

## Member 3: Nabila (World & Progression)

I handled the world generation and game progression systems. I implemented a minimap that reveals only explored areas, helping players navigate the dungeon. A level progression system was developed featuring locked doors that require keys found within the dungeon. Levers can also be used to unlock doors, and the exit becomes available after reaching it. I also added save/load functionality using file I/O, allowing players to persist their health, inventory, and level progress. The game supports both first-person and third-person camera views, which can be toggled with the 'V' key.

# System Design

## Coordinate System

X-axis represents horizontal east direction, Y-axis represents horizontal north direction, and Z-axis represents vertical height.

## Rendering Pipeline

Each frame undergoes a structured rendering process: clearing buffers, setting up the camera and fog effects, rendering dungeon geometry, enemies and items, overlaying HUD and minimap elements, and finally swapping buffers to display the frame.

# Controls

| Key | Action |
|---|---|
| WASD | Movement |
| Arrow Keys | Look around |
| Left Click | Attack |
| Right Click / Q | Switch weapon |
| E | Interact |
| V | 1st Person / 3D View |
| TAB | Toggle minimap |
| F5 | Save |
| F9 | Load |
| N | Next level |
| R | Restart |
| L | Sprinting |
| ESC | Exit |

# Future Improvements

Future enhancements may include integrating sound effects for improved immersion, adding environmental traps and puzzle elements, and further expanding enemy AI capabilities and dungeon complexity.

# Conclusion

This project successfully demonstrates the implementation of a fully functional 3D dungeon crawler using fundamental computer graphics techniques. It highlights procedural generation, real-time rendering, interaction systems, and AI behavior, all developed within the constraints of PyOpenGL. The collaborative effort ensured a balanced distribution of responsibilities and resulted in a complete and engaging application.